

Chapter 14

Disciplined Convex Programming

Algorithms for Optimization, 2nd ed. (2026)
Kochenderfer & Wheeler

Lecture Slides

Chapter Outline

- 1 Canonical Form of a DCP
- 2 Verification of Convexity
- 3 Canonicalization
- 4 Solving with CVXPY
- 5 Summary & Exercises

What Is Disciplined Convex Programming? I

Convex optimization is special: when the objective function is **convex** and all constraints define a **convex feasible region**, then every local minimum is automatically a *global* minimum. This guarantee is enormously valuable in practice because it means no matter which starting point a solver uses, it will converge to the true best answer — not merely a locally decent one.

The difficulty, however, is *knowing* whether a given problem is truly convex. For small hand-crafted examples this can be checked with calculus, but for realistic engineering or data-science problems with hundreds of interleaved functions, manual verification is error-prone and impractical to automate without a disciplined framework.

The Core Challenge

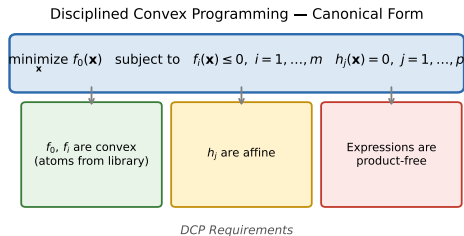
Checking convexity of an expression requires applying calculus rules (second-derivative tests, composition theorems, etc.) to every sub-expression in the problem. As problems grow in size and complexity, this becomes tedious and nearly impossible to do reliably by hand. We therefore need a *systematic, mechanical* procedure that a computer can execute automatically.

What Is Disciplined Convex Programming? II

DCP: Disciplined Convex Programming — Definition

A **disciplined convex program (DCP)** is a convex optimization problem that is written according to a fixed set of *grammatical rules*, so that automated software can (1) verify its convexity, and (2) convert it into a standard *canonical form* accepted by high-performance numerical solvers. The word “disciplined” means the problem writer follows the grammar; “programming” here means mathematical programming (optimisation), not computer programming.

The figure on the right illustrates the DCP workflow. A user writes a problem using DCP syntax; the system checks the grammar (verification); converts it to canonical form (canonicalization); and hands it to a solver. The key building blocks are *atoms* — a curated library of primitive mathematical functions whose convexity properties are pre-certified. Every expression in a DCP is built by composing atoms.



What Is Disciplined Convex Programming? III

Key Insight

DCP separates *modelling* (writing the problem in human-readable form) from *solving* (running numerical algorithms). The user never hand-derives gradients, reformulations, or solver-specific data structures; the DCP system handles all of that automatically and provably correctly.

Canonical Form of a Convex Program I

Every convex optimization problem, no matter how it is originally stated, can be cast into a *standard form* that unambiguously specifies the objective function and every constraint. This standard form is what the DCP framework targets and what solvers expect.

Standard Convex Problem — Eq. 14.1

$$\underset{\mathbf{x}}{\text{minimize}} \quad f_0(\mathbf{x}) \quad \text{subject to} \quad \begin{cases} f_i(\mathbf{x}) \leq 0 & i = 1, \dots, m \\ h_j(\mathbf{x}) = 0 & j = 1, \dots, p \end{cases}$$

Canonical Form of a Convex Program II

Notation — Every Symbol Explained

- $\mathbf{x} \in \mathbb{R}^n$ (“ $\mathbf{x}$ bold, in R- n ”) is the *decision variable*: an n -dimensional vector of real numbers that we are free to choose in order to minimise the objective. Think of each component $x_1, x_2, \dots, x_n$ as one “knob” we can turn.
- $f_0(\mathbf{x})$ (“ f -zero of $\mathbf{x}$ ”) is the *objective function*: the quantity we want to make as small as possible. DCP requires f_0 to be **convex** — meaning its graph looks like a bowl that opens upward, with no hidden valleys.
- $f_i(\mathbf{x})$ (“ f sub i of $\mathbf{x}$ ”, for $i = 1, \dots, m$) are the m *inequality constraint functions*. The requirement $f_i(\mathbf{x}) \leq 0$ says that the solution must lie inside the region where f_i is non-positive. DCP requires each f_i to also be **convex**, so the feasible region for that constraint is a convex set.
- $h_j(\mathbf{x})$ (“ h sub j of $\mathbf{x}$ ”, for $j = 1, \dots, p$) are the p *equality constraint functions*. These must be **affine** (of the form $\mathbf{a}^\top \mathbf{x} - b$, i.e., a linear function of $\mathbf{x}$). The reason is explained in the alert below.
- m is the total number of inequality constraints; p is the total number of equality constraints; n is the number of decision variables.

Canonical Form of a Convex Program III

How to Read This Formula

We are searching over all n -dimensional vectors $\mathbf{x}$ for the one that minimises $f_0(\mathbf{x})$ while staying inside the *feasible region* — the set of all $\mathbf{x}$ satisfying every constraint. Because every f_i is convex, each feasible region $\{f_i \leq 0\}$ is itself a convex set, and the intersection of convex sets is convex, so the full feasible region is convex. Minimising a convex function over a convex set is always a well-behaved problem: there is exactly one globally optimal value (though there may be multiple optimal points), and any local minimiser is a global minimiser.

Why Must Equality Constraints Be Affine?

An equality constraint $h(\mathbf{x}) = 0$ is logically equivalent to two simultaneous inequality constraints: $h(\mathbf{x}) \leq 0$ and $h(\mathbf{x}) \geq 0$. The first requires h to be **convex**; the second, rewritten as $-h(\mathbf{x}) \leq 0$, requires $-h$ to be convex, i.e., h to be **concave**. A function that is simultaneously convex and concave must be **affine**. This is why DCP forbids nonlinear equality constraints.

Canonical Form of a Convex Program IV

The Four DCP Requirements

A problem is a valid DCP only if it satisfies all four conditions:

- 1 **Product-free expressions.** No expression may contain the product of two non-constant sub-expressions. For instance, $x_1 \cdot x_2$ (where both factors vary) is forbidden. This restriction ensures curvature can be tracked symbolically through the expression.
- 2 **Top-level curvature rules.** The top-level role of each expression must match its curvature: a minimised objective must be convex, a maximised objective must be concave, an inequality left-hand side must be convex, and an equality must be affine.
- 3 **Sign rules.** The system tracks whether each constant coefficient is positive or negative, because multiplying a convex function by -1 turns it concave.
- 4 **Composition rules.** When one function is applied to the output of another, the DCP system checks whether the curvature and monotonicity (“increasing or decreasing”) of the outer function are compatible with the curvature of the inner expression.

Running Example — Verifying a Mixed-Norm Problem (Ex. 14.1) I

To make the abstract definitions concrete, let us trace through a specific problem step by step and verify that every DCP rule is satisfied. This running example will be used throughout the chapter.

The Problem to Verify

$$\underset{\mathbf{x}}{\text{minimize}} \|\mathbf{Ax} - \mathbf{b}\|_1 \quad \text{subject to } \|\mathbf{Cx} - \mathbf{d}\|_{1.5} \leq 3$$

with specific numerical matrices and vectors:

$$\mathbf{A} = \begin{bmatrix} 2 & -1 \\ 1 & 3 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 4 \\ -5 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 0 & 1 \\ 3 & -4 \end{bmatrix}, \quad \mathbf{d} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}$$

Running Example — Verifying a Mixed-Norm Problem (Ex. 14.1) II

Notation — Every Symbol in This Problem

- $\mathbf{x} \in \mathbb{R}^2$ (“ $\mathbf{x}$ bold in R-2”) is the two-dimensional decision vector with components x_1 and x_2 . This is the variable we will solve for.
- $\mathbf{A}$ and $\mathbf{C}$ are 2×2 matrices of *fixed* (known) numbers — they are not variables.
- $\mathbf{b}$ and $\mathbf{d}$ are fixed two-dimensional vectors.
- $\|\mathbf{v}\|_1$ (“L-one norm of $\mathbf{v}$ ”) of a vector $\mathbf{v} = (v_1, v_2)$ equals $|v_1| + |v_2|$, the sum of the absolute values of all components. More generally for n components: $\|\mathbf{v}\|_1 = \sum_{k=1}^n |v_k|$.
- $\|\mathbf{v}\|_{1.5}$ (“L-1.5 norm of $\mathbf{v}$ ”) is the L_p norm with $p = 1.5$, defined as $(\sum_k |v_k|^{1.5})^{1/1.5}$. Any L_p norm with $p \geq 1$ is a convex function, because $p = 1$ gives the convex L_1 norm and all larger p give “rounder” unit balls that are still convex.
- The constraint $\|\mathbf{C}\mathbf{x} - \mathbf{d}\|_{1.5} \leq 3$ says: “the $L_{1.5}$ distance between $\mathbf{C}\mathbf{x}$ and $\mathbf{d}$ must be at most 3.”

Running Example — Verifying a Mixed-Norm Problem (Ex. 14.1) III

Step-by-Step DCP Verification

We verify the problem by checking each piece against the DCP rules.

- 1 The expression $\mathbf{Ax} - \mathbf{b}$ is the product of a *fixed* matrix $\mathbf{A}$ with the variable vector $\mathbf{x}$, minus a constant. This is an **affine** (linear) function of $\mathbf{x}$. Affine functions are simultaneously convex and concave — the mildest possible curvature.
- 2 The L_1 norm $\|\cdot\|_1$ is a convex function listed in the atom library. Applying a convex norm to an affine argument gives a **convex** function — this follows from the composition rule (norms are nondecreasing in the absolute values of their inputs, and the affine argument satisfies the compatibility condition).
- 3 Similarly, $\mathbf{Cx} - \mathbf{d}$ is affine, and $\|\cdot\|_{1.5}$ is convex, so their composition is **convex**.
- 4 The constraint is of the form *convex function* $\leq$ *constant* (the number 3 is convex — all constants are both convex and concave), which satisfies the DCP inequality rule.
- 5 The objective asks to *minimise* a *convex* function, which is exactly what DCP requires.

Conclusion: this is a valid DCP.

Running Example — Verifying a Mixed-Norm Problem (Ex. 14.1) IV

Key Insight

Each check above is performed *mechanically* by software, consulting only the atom library for curvature information. The human writer never needs to invoke the mathematical definition of convexity directly. This is the entire point of the DCP framework.

The Atom Library — Building Blocks of DCP I

The DCP framework rests on a curated *atom library*: a catalogue of primitive mathematical functions for which the convexity properties have been pre-certified. When you write a DCP expression, you are composing atoms together according to grammar rules, exactly as you compose words according to language rules. As long as you stay within the grammar, convexity of the whole expression is guaranteed.

Atom Library — Common Functions

Function	Domain	Curvature	Monotonicity	Range
$ x $	$\mathbb{R}$	convex	nonmonotone	$[0, \infty)$
x^2	$\mathbb{R}$	convex	nonmonotone	$[0, \infty)$
$\max(x_1, x_2)$	$\mathbb{R}^2$	convex	nondecreasing	$\mathbb{R}$
$\exp(x)$	$\mathbb{R}$	convex	nondecreasing	$(0, \infty)$
$\log(x)$	$(0, \infty)$	concave	nondecreasing	$\mathbb{R}$
$\sqrt{x}$	$[0, \infty)$	concave	nondecreasing	$[0, \infty)$
$\min(x_1, x_2)$	$\mathbb{R}^2$	concave	nondecreasing	$\mathbb{R}$
$\ x\ _p$	$\mathbb{R}^n$	convex	nonmonotone	$[0, \infty)$

What Each Atom Records

For each function f in the library, four pieces of information are stored:

- **Domain** — the set of inputs for which f is defined. For example, $\log(x)$ requires $x > 0$.
- **Curvature** — whether f is convex, concave, or affine (linear) over its domain.
- **Monotonicity** (per argument) — whether f is nondecreasing or nonincreasing in each of its arguments. This information is essential for the

The Atom Library — Building Blocks of DCP II

Common Atoms and Their Properties

Atom	Mathematical notation	Curvature	Monotonicity	Domain
Absolute value	$ x $	Convex	Nonincreasing for $x < 0$; nondecreasing for $x > 0$	$\mathbb{R}$
Square	x^2	Convex	Same as $ x $	$\mathbb{R}$
Square root	$\sqrt{x}$	Concave	Nondecreasing	$x \geq 0$
Natural log	$\log(x)$	Concave	Nondecreasing	$x > 0$
Exponential	e^x	Convex	Nondecreasing	$\mathbb{R}$
L_p norm	$\ \mathbf{v}\ _p$	Convex	— (depends on sign of each component)	$\mathbb{R}^n$
Maximum	$\max(a, b)$	Convex	Nondecreasing in both	$\mathbb{R}$

The Atom Library — Building Blocks of DCP III

Sets as Atoms

Convex *sets* can also be treated as atoms. Some common examples:

- The non-negative orthant $\mathbb{R}_+^n = \{\mathbf{x} : x_i \geq 0 \text{ for all } i\}$ (“R-n-plus”) is a convex set, used to represent non-negativity constraints on all variables.
- The positive semidefinite cone $\mathbb{S}_+^n = \{X \in \mathbb{R}^{n \times n} : X = X^\top, \mathbf{v}^\top X \mathbf{v} \geq 0 \text{ for all } \mathbf{v}\}$ (“S-n-plus”, meaning all symmetric matrices with non-negative curvature) is used in semidefinite programming.
- The *epigraph* (“epi”) and *hypograph* (“hypo”) of a function (defined in Section 3) are convex sets closely associated with convex and concave functions respectively.

Key Insight

The atom library is the heart of the DCP system. Adding a new atom (with its certified curvature information) immediately extends the class of problems that DCP can handle, without any changes to the verification or canonicalization algorithms.

Verification — The Two-Stage Pipeline I

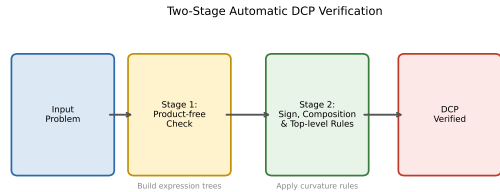
Once a problem is written in DCP notation, the software must determine whether it actually satisfies the convexity requirements. This verification is done in *two sequential stages*: a syntactic stage that checks the structure of the formula, and a semantic stage that propagates curvature labels.

Two-Stage Verification

- 1 **Stage 1 — Product-Free Check.** The system scans every expression and confirms that no expression contains the product of two non-constant sub-expressions. This stage is *purely syntactic*: it does not reason about curvature at all, only about the algebraic structure of the formula. It is fast and can be done in a single pass over the expression.
- 2 **Stage 2 — Rule Checking.** The system constructs an *expression tree* — a tree-shaped representation where leaves are variables or constants and internal nodes are operations or atoms — and then walks the tree from the leaves (bottom) up to the root (top), assigning a curvature label (*convex* / *concave* / *affine* / *neither*) to each node, and verifying the sign, composition, and top-level production rules at each step.

Verification — The Two-Stage Pipeline II

Both stages are fully automatic. If Stage 1 fails, the system reports which sub-expression contains a forbidden product and stops. If Stage 2 finds a curvature violation, it reports the offending node in the tree so the user can fix the formulation. If both stages pass, the problem is a valid DCP and proceeds to canonicalization.



Key Insight

Verification runs in time proportional to the size of the expression (i.e., the number of operations), so it is essentially instantaneous even for large problems. The cost of the verification step is negligible compared to the cost of solving the problem.

Product-Free Expressions — The First Filter I

The product-free requirement is the first filter in DCP verification. Its purpose is to ensure that every expression can be written as a *linear combination* of atom outputs, which makes it possible to track curvature symbolically through the formula.

Definition: Product-Free Expression (Eq. 14.5)

An expression e is called **product-free** if and only if it can be written in the form:

$$e = a + \mathbf{b}^\top \mathbf{x} + \sum_{j=1}^q c_j f_j(\mathbf{e}_j)$$

Product-Free Expressions — The First Filter II

Notation — Every Symbol Explained

- $a \in \mathbb{R}$ (“a in R”) is a real-valued *constant*: a fixed number that does not depend on any variable.
- $\mathbf{b} \in \mathbb{R}^n$ (“b bold in R-n”) is a *constant* vector of coefficients.
- $\mathbf{b}^\top \mathbf{x}$ (“b-transpose times x”, also written as the inner product $\langle \mathbf{b}, \mathbf{x} \rangle$) equals $b_1x_1 + b_2x_2 + \dots + b_nx_n$, which is an affine (linear) function of $\mathbf{x}$. The $\top$ symbol (“superscript T” or “transpose”) turns the column vector $\mathbf{b}$ into a row vector so that the multiplication is defined.
- $\sum_{j=1}^q$ (Σ (Sigma), meaning “sum from j equals 1 to q ”) denotes the sum of q terms, one for each atom call in the expression.
- c_j is a *constant* scalar coefficient for the j -th atom call. It can be positive, negative, or zero, but it must not involve any variable.
- f_j (“f sub j”) is the j -th atom from the atom library (for example, $\log$, $\|\cdot\|_2$, $\max$, $e^{(\cdot)}$).
- $\mathbf{e}_j$ (“e bold sub j”) is the vector of sub-expressions passed as arguments to atom f_j . Crucially, each component of $\mathbf{e}_j$ must itself be a product-free expression — the definition is *recursive*. This recursion is what allows complex nested expressions to be classified.

Product-Free Expressions — The First Filter III

How to Read This Definition

A product-free expression is built from: (a) a constant a , plus (b) an affine (linear) term in the decision variables $\mathbf{x}$, plus (c) a weighted sum of atom evaluations, where the weights c_j are all constants. The only operations allowed are **addition**, **subtraction**, and **scalar multiplication by constants**. No variable-times-variable products are permitted. This restriction is what makes symbolic curvature tracking possible: the system only needs to look up the curvature of each atom f_j in the library, and the rest is just addition with known signs.

Valid (Product-Free) Examples

- $3 + 2x_1 + \log(x_2)$: constant 3, linear term $2x_1$, atom $\log(x_2)$ — product-free because 2 is a constant multiplier, not a variable.
- $\max(x_1^2 - 3, x_2)$: the max atom applied to two sub-expressions, where x_1^2 is handled by the square atom — product-free.
- $\|\mathbf{Ax} - \mathbf{b}\|_2$: the L_2 -norm atom applied to an affine expression ($\mathbf{A}$ is a fixed matrix) —

Invalid (Not Product-Free) Examples

- $x_1 \cdot x_2$: product of two variables — rejected.
- x_1^2 written as $x_1 \times x_1$: same issue. However, x_1^2 is allowed if written as $\text{square}(x_1)$, using the designated atom with its pre-certified curvature.
- $\sin(x_1) \cdot \log(x_2)$: product of two non-constant sub-expressions — rejected.

Important Note on x^2

The expression x^2 looks like a product, but the DCP system handles it through the square atom, which stores the curvature information directly: convex, nondecreasing for $x \geq 0$, nonincreasing for $x \leq 0$, with range $[0, \infty)$. The atom library is the bridge that allows nonlinear functions while still maintaining the product-free property. The key principle: any nonlinear function you use must appear as a named atom in the library, not as an explicit multiplication of variables.

Sign Rules — How Scaling Changes Curvature I

Once an expression passes the product-free check, the system assigns curvature labels by applying *sign rules*. These rules govern how multiplying by a constant scalar, or adding two expressions together, affects whether the result is convex, concave, or affine.

The intuition is geometric: a convex function looks like a bowl. Multiplying it by a positive constant scales the bowl but keeps it pointing upward (convex). Multiplying by a negative number flips the bowl upside down, turning it into an arch (concave).

Sign Rules — How Scaling Changes Curvature II

Sign Rules for Scaling by a Constant

Let $f(\mathbf{x})$ be a function with known curvature, and let c be a constant real number (positive or negative). Then:

- If f is **convex** and $c > 0$, then $c \cdot f(\mathbf{x})$ is **convex**. Scaling a bowl upward keeps it a bowl.
- If f is **convex** and $c < 0$, then $c \cdot f(\mathbf{x})$ is **concave**. Flipping a bowl upside down gives an arch.
- If f is **concave** and $c > 0$, then $c \cdot f(\mathbf{x})$ is **concave**. Scaling an arch keeps it an arch.
- If f is **concave** and $c < 0$, then $c \cdot f(\mathbf{x})$ is **convex**. Flipping an arch gives a bowl.

The pattern is: the curvature *flips* when you multiply by a negative constant, and *stays the same* for a positive constant.

Sign Rules — How Scaling Changes Curvature III

Sum Rules for Adding Two Functions

Adding two functions affects curvature as follows:

- $\text{convex} + \text{convex} = \textbf{convex}$. Two bowls added together give a bowl.
- $\text{concave} + \text{concave} = \textbf{concave}$. Two arches added give an arch.
- $\text{affine} + \text{convex} = \textbf{convex}$. A flat plate added to a bowl gives a bowl.
- $\text{affine} + \text{concave} = \textbf{concave}$.
- $\text{convex} + \text{concave} = \textbf{neither}$ (in general). A bowl plus an arch can give almost any shape.
- $\text{affine} + \text{affine} = \textbf{affine}$. Two flat plates sum to a flat plate.

Sign Rules — How Scaling Changes Curvature IV

Worked Example: Classifying $3 + 2 \log(1 + x_1) - \sin(x_2)$

We classify this expression step by step, starting from the innermost sub-expressions. We assume $x_1 \geq 0$ (so $1 + x_1 \geq 1$, keeping $\log$ in its domain) and $x_2 \in [-\pi, 0]$.

Step 1. The natural logarithm $\log(1 + x_1)$ is a **concave** function. This is an atom in the library with known curvature.

Step 2. Multiplying by the positive constant 2: $2 \log(1 + x_1)$ is **concave** (positive constant times concave stays concave by the sign rule).

Step 3. The sine function $\sin(x_2)$ on $x_2 \in [-\pi, 0]$ has a non-negative second derivative on this interval (specifically $-\sin(x_2) \geq 0$ there), so $\sin(x_2)$ is **convex** on $[-\pi, 0]$.

Step 4. The coefficient of $\sin(x_2)$ in the full expression is -1 (negative), so $-\sin(x_2)$ is **concave** (negative constant times convex is concave).

Step 5. Adding:

$$\underbrace{3}_{\text{affine}} + \underbrace{2 \log(1 + x_1)}_{\text{concave}} + \underbrace{(-\sin(x_2))}_{\text{concave}}$$

Composition Rules — Chaining Functions Together I

The composition rule handles the most common pattern in practice: applying one function to the output of another. For example, $1/g(\mathbf{x})$ composes the reciprocal function $f(t) = 1/t$ with $g(\mathbf{x})$. The composition rule tells us when $f(g(\mathbf{x}))$ is convex or concave, based on the properties of f and g individually.

The intuition: think of curvature as “direction” (convex = curving up, concave = curving down). If the outer function is nondecreasing (it does not reverse direction), then the curvature of the inner function propagates unchanged. If the outer function is nonincreasing (it reverses direction), then the curvature of the inner function flips.

Composition Rules — Chaining Functions Together II

Composition Theorem (Eqs. 14.3–14.4)

The composed function $f(\varepsilon_1, \dots, \varepsilon_m)$, where ε (epsilon) denotes each sub-expression argument, is **convex** if all of the following hold:

- f is itself either affine or **convex**; and
- For each argument ε_i (“epsilon sub i”, the i -th argument to f), one of the three cases applies:
 - ▷ f is *nondecreasing* (i.e., larger input gives larger output) in ε_i over the range $\mathcal{R}$ (“ $\mathcal{R}$, the effective range”) of ε_i , and ε_i is **convex**; or
 - ▷ f is *nonincreasing* (larger input gives smaller output) in ε_i over $\mathcal{R}$, and ε_i is **concave**; or
 - ▷ ε_i is **affine** (in which case no monotonicity condition on f is needed, because affine inputs are both convex and concave).

Symmetric rules apply for establishing **concavity**.

Composition Rules — Chaining Functions Together III

How to Read This Theorem

Think of “convex” as “curving up” and “concave” as “curving down”. A nondecreasing outer function passes curvature through unchanged. A nonincreasing outer function flips the curvature (just as multiplying by a negative number flips the sign). So: nondecreasing outer + convex inner = convex (up through up stays up); nonincreasing outer + concave inner = convex (down then flipped = up). Two “flips” cancel, preserving convexity.

Composition Rules — Chaining Functions Together IV

Decomposition Consequence (Eq. 14.7)

When f is convex and nondecreasing, the single constraint $f(g(\mathbf{x})) \leq y$ can be split into two simpler constraints using a fresh auxiliary variable z (“z”):

$$f(g(\mathbf{x})) \leq y \quad \Longleftrightarrow \quad \begin{cases} f(z) \leq y \\ g(\mathbf{x}) \leq z \end{cases}$$

This splitting is valid because f is nondecreasing: if $g(\mathbf{x}) \leq z$ then $f(g(\mathbf{x})) \leq f(z) \leq y$. This splitting technique — introducing a fresh variable for each function application — is the basis of the canonicalization step described in Section 3.

Composition Rules — Worked Examples I

Two detailed worked examples apply the composition theorem to verify whether specific composed expressions are convex.

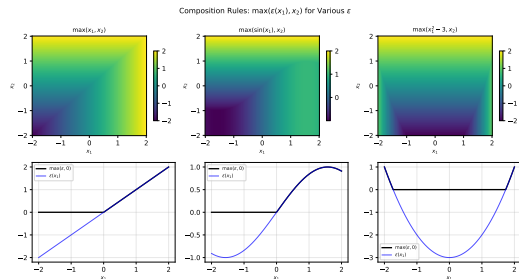
Ex. 14.3: Is $f(g(\mathbf{x})) = 1/g(\mathbf{x})$ convex?

Here $f(t) = 1/t$ (the reciprocal function) and $g(\mathbf{x}) = 3 + 2\log(1 + x_1) - \sin(x_2)$, the expression we already classified as **concave** with range contained in $[3, \infty)$ (since the minimum value of g is 3, achieved when $\log$ and the correction from $\sin$ are both at their minimum).

Step 1. From the previous example, g is **concave** with effective range $\mathcal{R} \subseteq [3, \infty)$.

Step 2. The outer function $f(t) = 1/t$ is **convex and nonincreasing** for $t > 0$ (as t grows, $1/t$ decreases; as t shrinks toward 0, $1/t$ blows up convexly).

Step 3. Apply the composition theorem: f is convex



Ex. 14.5: Is $\max(\varepsilon(x_1), x_2)$ convex?

The $\max$ function is convex and nondecreasing in both arguments. For the first argument ε :

- $\varepsilon = x_1$ (affine): $\max(x_1, x_2)$ is **convex** (affine argument, third sub-case).

- $\varepsilon = \sin(x_1)$ (neither convex nor concave on

Composition Rules — Worked Examples II

Key Insight

The composition rule requires checking *both* the curvature of the outer function *and* its monotonicity over the specific range of the inner expression. Both conditions must be satisfied simultaneously. If either fails, the DCP system cannot certify convexity for that composition.

Top-Level Production Rules I

After the sign and composition rules have assigned curvature labels to all sub-expressions in the tree, the DCP system checks the *top-level role* of each expression. Each role — objective to minimise, objective to maximise, inequality constraint, equality constraint — has a required curvature. This is the final gate before a problem is declared valid.

Top-Level Curvature Requirements

Context	Required curvature	Concrete example
Minimise $f_0(\mathbf{x})$	f_0 must be convex	$\min \ \mathbf{x}\ _2$
Maximise $f_0(\mathbf{x})$	f_0 must be concave	$\max \log(\det X)$
Inequality $f(\mathbf{x}) \leq c$	f must be convex ; c must be concave (or constant)	$\ g(\mathbf{x})\ _2 \leq 3$
Inequality $c \leq f(\mathbf{x})$	c must be convex (or constant); f must be concave	$0 \leq \log(\mathbf{x})$
Equality $h(\mathbf{x}) = 0$	h must be affine	$\mathbf{Ax} = \mathbf{b}$

Top-Level Production Rules II

Why These Rules Follow from Convex Analysis

Each rule is a consequence of convex analysis:

- Minimising a convex function gives a problem with a unique global minimum (or a convex set of equally-good minima).
- Maximising a concave function is equivalent (by multiplying by -1) to minimising a convex function — same guarantee.
- An inequality $f(\mathbf{x}) \leq c$ with f convex defines a convex sublevel set, i.e., a convex feasible region.
- An equality $h(\mathbf{x}) = 0$ simultaneously requires $h \leq 0$ (convex) and $-h \leq 0$ (also convex, i.e., h concave), so h must be affine — the only functions that are both convex and concave.

Top-Level Production Rules III

Example: Nonlinear Equality Is Rejected

Suppose someone writes $x_1^2 + x_2^2 = 1$ (the unit circle) as an equality constraint. The function $f(\mathbf{x}) = x_1^2 + x_2^2$ is convex (a bowl) but *not* concave (not an arch), so it cannot satisfy both inequality requirements simultaneously. The DCP system will reject this with an informative error message. The correct way to handle such a constraint is to relax it to an inequality $x_1^2 + x_2^2 \leq 1$ (a disk, which is convex) or to reformulate the problem entirely.

Automatic Verification — Expression Tree Walkthrough (Ex. 14.6) I

The expression tree is the internal data structure used by DCP software to represent a formula. Each leaf node is a constant or variable; each internal node is an operation or atom; and the root represents the whole expression. Verification consists of a single bottom-up pass that assigns curvature labels as it goes.

Problem (Ex. 14.6)

$$\underset{x_1, x_2}{\text{minimize}} \quad \text{pow}(a_1 x_1 + a_2 x_2 - b_1, 2)$$

$$\text{s.t.} \quad c_1 x_1 + c_2 x_2 = d, \quad \sqrt{\min(x_1, x_2)} \geq 1 - \log(x_2)$$

where $a_1, a_2, b_1, c_1, c_2, d$ are known constants.

Curvature Trace (Bottom-Up)

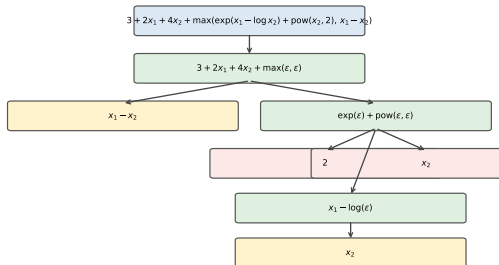
minimize

$\hookrightarrow \text{pow}(\varepsilon, 2)$: **convex** — squaring is convex

$\hookrightarrow -b_1 + a_1 x_1 + a_2 x_2$: **affine** — linear in x_1, x_2

equality constraint:

Expression Tree for DCP Verification (Fig 14.2)



Domain Conditions Inferred Automatically

As the tree is traversed, the system automatically collects domain conditions implied by the atoms.

Key Insight

The expression tree makes verification both *complete* (every sub-expression is inspected) and *efficient* (each node is visited exactly once, so the total work is proportional to the number of operations in the problem — essentially instantaneous). This is far more reliable than manual convexity checking and does not require the user to understand the underlying mathematics.

Canonicalization — What It Is and Why It Matters I

Once the DCP system has verified that the problem is convex, it performs **canonicalization**: a fully automatic transformation of the problem from the user's high-level, expressive form into a *partitioned canonical form* that standard solvers can directly accept. This transformation is always mathematically correct — the canonical form has exactly the same optimal solution as the original problem.

Why Canonical Form Is Needed

Different convex solvers (SCS, ECOS, MOSEK, Clarabel, etc.) each accept problems in their own specific internal format. By converting any valid DCP into a single canonical form, the CVXPY library can serve as a *universal translation layer*: the user writes in expressive DCP notation, and the backend automatically generates the solver-specific input data. This also enables the system to extract *dual information* (Lagrange multipliers, sensitivity analysis, optimality certificates) in a uniform way.

Partitioned Canonical Form (Eq. 14.8)

$$\begin{aligned} & \underset{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(k)}}{\text{minimize}} && d + \sum_{i=1}^k (\mathbf{c}^{(i)})^\top \mathbf{x}^{(i)} \\ & \text{subject to} && \mathbf{x}^{(i)} \in S^{(i)} \quad (i = 1, \dots, k), \quad \sum_{i=1}^k \mathbf{A}^{(i)} \mathbf{x}^{(i)} = \mathbf{b} \end{aligned}$$

Canonicalization — What It Is and Why It Matters III

Notation — Every Symbol in the Canonical Form

- $\mathbf{x}^{(i)}$ (“**x** bold superscript *i*”, meaning the *i*-th block of variables) is one *block* of decision variables. The original problem’s variables, plus all auxiliary variables introduced during canonicalization, are split into *k* separate blocks. The superscript here is a block index, not a power.
- d is a real constant (a fixed numerical offset in the objective, which has no effect on the optimal solution but is needed to report the correct optimal value).
- $\mathbf{c}^{(i)}$ (“**c** bold superscript *i*”) is the linear cost vector for block *i*. The objective is a simple **linear** (dot product) combination of all variables.
- $S^{(i)}$ (“**S** superscript *i*”) is a **convex cone** (or convex set) from the atom library — for example, the non-negative orthant, the second-order cone, or the positive semidefinite cone. Membership in $S^{(i)}$ encodes all the nonlinear structure of the original problem in a form solvers understand natively.
- $\mathbf{A}^{(i)}$ (“**A** bold superscript *i*”) is a matrix of coupling coefficients. The equality $\sum_{i=1}^k \mathbf{A}^{(i)} \mathbf{x}^{(i)} = \mathbf{b}$ links the blocks together. Outside this one equality, all blocks are completely independent.

Canonicalization — What It Is and Why It Matters IV

How to Read the Canonical Form

The objective is purely **linear** in all variables — there is no quadratic or other nonlinear term. All nonlinearity is hidden inside the set-membership constraints $\mathbf{x}^{(i)} \in S^{(i)}$. The coupling equality $\sum_i \mathbf{A}^{(i)} \mathbf{x}^{(i)} = \mathbf{b}$ is the only link between blocks. This block structure causes the Hessian (matrix of second derivatives) of the barrier function used by interior point solvers to be **block-diagonal**, which is far cheaper to invert than a dense matrix — enabling dramatic speed improvements for large problems.

Two Steps of Canonicalization

Canonicalization proceeds in two steps:

- 1 **Linearization.** Each atom call $f_j(\mathbf{e}_j)$ in every expression is replaced by a fresh scalar variable v_j , and a constraint is added that ties v_j to the atom's value. After this step, the objective and all constraint expressions are *affine* (linear) in the expanded set of original and auxiliary variables.
- 2 **Graph implementation.** Each atom constraint (e.g., $f(u) \leq v$) is further expanded into its *graph form* — a set of linear or conic (second-order cone, semidefinite, etc.) constraints that characterise the epigraph or hypograph of the atom. After this step, all constraints are in standard conic form.

Linearization — Replacing Atoms with Variables I

Linearization is the first step of canonicalization. Its goal is to replace every nonlinear atom call with a fresh auxiliary variable, making all expressions in the problem linear (affine) in the expanded variable set. The nonlinearity is moved from the expressions into new constraints.

Linearization — Replacing Atoms with Variables II

Linearization Process (Eqs. 14.9–14.11)

Starting from the product-free expression:

$$a + \mathbf{b}^\top \mathbf{x} + \sum_{j=1}^q c_j f_j(\mathbf{e}_j)$$

we introduce one new *atom variable* v_j for each atom call. The expression becomes the affine function:

$$a + \mathbf{b}^\top \mathbf{x} + \mathbf{c}^\top \mathbf{v}$$

where $\mathbf{v} = (v_1, \dots, v_q)$ is the vector of new atom variables, and $\mathbf{c} = (c_1, \dots, c_q)$ collects the original coefficients.

For each $j = 1, \dots, q$, the following constraint is added to tie v_j to the actual atom value:

$$\begin{cases} f_j(\mathbf{e}_j) \leq v_j & \text{if } f_j \text{ is } \mathbf{convex} \\ f_j(\mathbf{e}_j) \geq v_j & \text{if } f_j \text{ is } \mathbf{concave} \end{cases}$$

Linearization — Worked Numerical Example (Ex. 14.8) I

Let us trace through the complete linearization of one concrete expression, step by step, to see exactly how the process introduces new variables and constraints.

Linearization — Worked Numerical Example (Ex. 14.8) II

Linearizing: $\text{pow}(3 + 2 \log(1 + x_1) - \sin(x_2), -1)$

This expression computes the reciprocal (i.e., $1/z$, where $z = 3 + 2 \log(1 + x_1) - \sin(x_2)$) of the expression we have been studying.

Step 1 — Identify the innermost affine arguments. The innermost sub-expressions are $1 + x_1$ (argument to $\log$) and x_2 (argument to $\sin$). Both are affine in x_1 and x_2 , so no new variables are needed at this level.

Step 2 — Handle the intermediate layer. The middle expression $3 + 2 \log(1 + x_1) - \sin(x_2)$ contains two atom calls: $\log$ (concave) and $\sin$ (convex on $[-\pi, 0]$). Introduce atom variables v_1 (for $\log$) and v_2 (for $\sin$), plus an argument alias $u_{1,1} = 1 + x_1$:

$$3 + 2v_1 - v_2 \quad \text{subject to:} \quad \log(u_{1,1}) \geq v_1, \quad u_{1,1} = 1 + x_1, \quad \sin(x_2) \leq v_2$$

The $\geq$ for $\log$ reflects that it is concave (hypograph variable); the $\leq$ for $\sin$ reflects that it is convex (epigraph variable).

Step 3 — Handle the outer layer. The atom $\text{pow}(\cdot, -1)$ (the reciprocal, which is convex for positive arguments) is applied to the middle expression. Introduce variable v_3 for the reciprocal and auxiliary argument variable $u_{3,1}$:

Graph Implementations — The Absolute Value I

After linearization, each atom constraint has the form $f(u) \leq v$ or $f(u) \geq v$, where u is affine. The second step of canonicalization replaces each such constraint with its *graph implementation*: a set of simpler linear or conic constraints that exactly characterise the same relationship without using the atom symbol f directly.

The name “graph implementation” comes from the fact that the set $\{(u, v) : f(u) \leq v\}$ is the *epigraph* of f — the region on or above the graph of the function — and we are implementing this set using basic linear or cone constraints.

Graph Implementations — The Absolute Value II

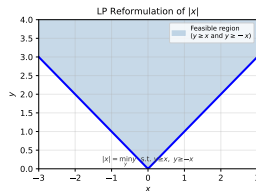
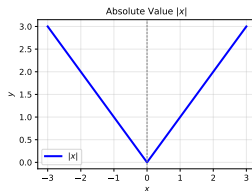
Graph Expansion of $|x|$ (Eq. 14.13)

The absolute value $|x|$ is a nonlinear function (it has a kink at $x = 0$ and is not differentiable there).

However, it can be characterised exactly as the smallest y that is simultaneously at least x and at least $-x$:

$$|x| \Rightarrow \begin{cases} \min_y y \\ \text{s.t. } y \geq x, \quad y \geq -x \end{cases}$$

To see why: if $x \geq 0$, then the binding constraint is $y \geq x$, giving $y = x = |x|$. If $x < 0$, then the binding constraint is $y \geq -x$, giving $y = -x = |x|$. Both constraints are linear in (x, y) , so the absolute value atom has been replaced by two linear inequalities. The kink disappears!



Epigraph and Hypograph — Geometry of Graph Implementations I

The absolute value example generalises: every graph implementation is derived from the *epigraph* (for convex atoms) or the *hypograph* (for concave atoms) of the function. These are geometric objects that capture the “above the graph” and “below the graph” regions.

Epigraph and Hypograph — Geometry of Graph Implementations II

Definitions (Eq. 14.15)

The **epigraph** (from the Greek “epi” meaning “above”) of a function f is the set of all points that lie on or above the graph of f :

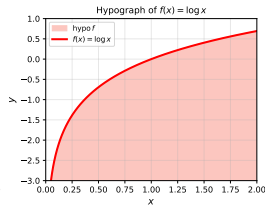
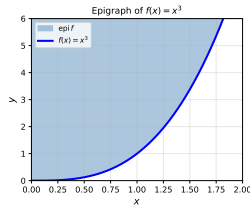
$$\text{epi } f = \{(\mathbf{x}, y) \in \mathbb{R}^n \times \mathbb{R} \mid y \geq f(\mathbf{x})\}$$

The **hypograph** (from the Greek “hypo” meaning “below”) of f is the set of all points that lie on or below the graph:

$$\text{hypo } f = \{(\mathbf{x}, y) \in \mathbb{R}^n \times \mathbb{R} \mid y \leq f(\mathbf{x})\}$$

The key mathematical facts are:

- f is **convex** $\iff$ $\text{epi } f$ is a **convex set**.
- f is **concave** $\iff$ $\text{hypo } f$ is a **convex set**.



Blue region: epigraph of x^3 on $x \geq 0$ (convex function, convex region above the curve). Red

Graph Implementations — L_1 , L_∞ , L_p , and L_2 Norms I

Norms are among the most frequently used atoms in DCP. Here we show how four important norms are each converted to solver-ready form through their graph implementations. In each case, the potentially complex norm expression becomes a small LP or conic programme.

Graph Implementations — L_1 , L_∞ , L_p , and L_2 Norms II

L_1 Norm as a Linear Programme (Ex. 14.9)

The L_1 norm $\|\mathbf{x}\|_1 = |x_1| + |x_2| + \cdots + |x_n|$ sums the absolute values of all components. Its graph expansion bounds each component independently with its own auxiliary variable:

$$\|\mathbf{x}\|_1 \Rightarrow \begin{cases} \min_{\mathbf{v}} \mathbf{1}^\top \mathbf{v} \\ \text{s.t. } -\mathbf{v} \leq \mathbf{x} \leq \mathbf{v} \end{cases}$$

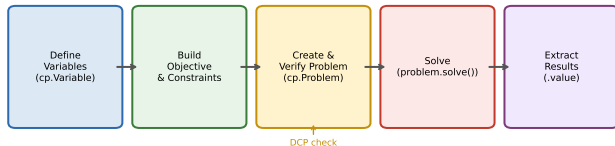
Here $\mathbf{v}$ (“ $\mathbf{v}$ bold”) is a vector of n auxiliary variables, one per component. The all-ones vector $\mathbf{1}$ (“ones vector”) makes the objective the sum $v_1 + v_2 + \cdots + v_n$. The component-wise constraint $-v_i \leq x_i \leq v_i$ means $v_i \geq |x_i|$, and since we minimise the sum, at optimality $v_i = |x_i|$ exactly.

L_∞ Norm as a Linear Programme

CVXPY is the standard Python library for disciplined convex programming. It is the Python counterpart of the Julia library `Convex.jl` used in the textbook. CVXPY implements the full DCP pipeline — verification, canonicalization, solver invocation, and result extraction — behind a clean and user-friendly Python interface. You write the problem in a natural mathematical notation, and CVXPY handles everything else.

CVXPY — Overview and Workflow II

CVXPY Workflow for Disciplined Convex Programming



What CVXPY Does Internally

- 1 **Parse:** CVXPY builds an expression graph (an internal tree representation) from the Python code you write. Each `cp.*` call creates a node.
- 2 **Verify:** runs the two-stage DCP check on the expression graph. If verification fails, an informative error message is raised before any numerical work is done.
- 3 **Canonicalize:** performs linearization and graph

Python vs. Julia

The textbook uses `Convex.jl` (Julia). Python users should use **CVXPY** instead — the API is nearly identical.

To install, run in your terminal:

```
pip install cvxpy
```

CVXPY ships with open-source solvers (SCS, ECOS) bundled; commercial solvers (MOSEK, Gurobi) can be added separately for larger or specialised problems.

Problem Types CVXPY Supports

- **LP** — Linear Programme
- **QP** — Quadratic Programme

CVXPY: Solving the Running Example (Ex. 14.1)

```
import cvxpy as cp
import numpy as np

#      Fixed data matrices and vectors

A = np.array([[2, -1], [1, 3]])      # 2x2 matrix (fixed)
b = np.array([4, -5])                # 2-vector (fixed)
C = np.array([[0, 1], [3, -4]])      # 2x2 matrix (fixed)
d = np.array([-1, 0])                # 2-vector (fixed)

#      Decision variable

x = cp.Variable(2)    # x is a 2-dimensional vector variable

#      Objective: minimise the L1 norm of A @ x - b
# cp.norm(expr, 1) computes the L1 norm (sum of absolute values)
objective = cp.Minimize(cp.norm(A @ x - b, 1))

#      Constraint: L1.5 norm of C @ x - d must be <= 3
# cp.norm(expr, 1.5) computes the L_{1.5} norm (convex for p>=1)
constraints = [cp.norm(C @ x - d, 1.5) <= 3]

#      Assemble and solve

problem = cp.Problem(objective, constraints)
```

CVXPY: Checking DCP Compliance Before Solving

```
import cvxpy as cp
import numpy as np

x = cp.Variable(2)
A = np.array([[2, -1], [1, 3]])
b = np.array([4., -5.])
C = np.array([[0., 1.], [3., -4.]])
d = np.array([-1., 0.])

objective = cp.Minimize(cp.norm(A @ x - b, 1))
constraint = cp.norm(C @ x - d, 1.5) <= 3
problem = cp.Problem(objective, [constraint])

# Check DCP compliance BEFORE calling solve().
# is_dcp() returns True if the problem satisfies all DCP rules.
# If False, calling solve() will raise a DCPError with details.
print("Is DCP?           ", problem.is_dcp())           # True
print("Obj convex?       ", objective.expr.is_convex()) # True
print("Constraint DCP?   ", constraint.is_dcp())         # True
print("Num variables:    ", len(problem.variables()))
print("Num constraints:  ", len(problem.constraints))

# Solve with a specific solver; ECOS is good for small SOCP
problem.solve(solver=cp.ECOS)
print(f"Optimal x: {x.value}")
```

Listing 2: Querying DCP properties of a problem before calling solve()

CVXPY: Lasso Regression — Sparse Recovery

```
import cvxpy as cp
import numpy as np

np.random.seed(42)
n, m = 100, 30          # n=100 samples (rows), m=30 features (cols)
A = np.random.randn(n, m) # random design matrix
# True coefficient vector: most entries are zero (sparse)
x_true = np.random.randn(m) * (np.random.rand(m) > 0.7)
# Observations: A @ x_true plus small Gaussian noise
b = A @ x_true + 0.1 * np.random.randn(n)

# lambda (lambda): regularisation weight, controls sparsity.
# Large lambda -> more zeros; small lambda -> closer least-squares fit.
lam = 0.5
x = cp.Variable(m)      # m-dimensional decision variable

# Objective = data fidelity term + sparsity penalty:
# ||Ax - b||^2 : least-squares fit (convex, differentiable)
# lambda * ||x||_1 : L1 penalty promotes sparsity (convex)
objective = cp.Minimize(
    cp.sum_squares(A @ x - b) + lam * cp.norm1(x)
)
problem = cp.Problem(objective)
print("Is DCP?:", problem.is_dcp()) # True
problem.solve()

# How many components of x are nonzero (threshold at 0.01)?
```

CVXPY: Mean-Variance Portfolio Optimisation

```
import cvxpy as cp
import numpy as np

np.random.seed(0)
n = 10 # number of assets
# mu (mu, pronounced "myoo"): vector of expected returns per asset
mu = np.random.randn(n) * 0.1 + 0.05
# Sigma (Sigma, capital Sigma): covariance matrix of asset returns.
# Positive semidefinite, so cp.quad_form(w, Sig) is convex.
Sig = np.random.randn(n, n)
Sig = Sig.T @ Sig / n + 0.01 * np.eye(n)
# gamma (gamma, pronounced "gam-ma"): risk-aversion coefficient.
# Large gamma -> more risk-averse; small gamma -> chase return.
gamma = 1.0

w = cp.Variable(n) # w: portfolio weight vector (sums to 1)

# cp.quad_form(w, Sig) = w^T Sigma w = portfolio variance (convex)
# mu @ w = mu^T w = expected return (affine in w)
risk = cp.quad_form(w, Sig) # convex (Sig is PSD)
ret = mu @ w # affine in w
# Objective: minimise variance - gamma * expected_return.
# This trades off lower risk against higher return.
objective = cp.Minimize(risk - gamma * ret)
constraints = [
    cp.sum(w) == 1, # budget: all wealth must be invested
    w >= 0 # long-only: no short selling (no borrowing)
```

Interior Point Methods and Block-Diagonal Structure I

Once the DCP has been converted to partitioned canonical form, it is handed to a numerical solver. The solver of choice for convex programmes is an **interior point method** (also called a *barrier method*). Understanding why interior point methods are so efficient for the canonical form helps explain why DCP is not just a notational convenience but a genuine algorithmic advantage.

How Interior Point Methods Work

An interior point method avoids constraint boundaries by adding a *logarithmic barrier function* to the objective that becomes $+\infty$ at the boundary of the feasible set. At each iteration, it solves a smooth unconstrained problem (the barrier-penalised version), then decreases the barrier weight. As the weight shrinks to zero, the iterates approach the true constrained optimum from the interior of the feasible set. Each unconstrained solve requires inverting (or factoring) the Hessian ∇^2 (nabla-squared, meaning the matrix of second derivatives) of the barrier function.

Interior Point Methods and Block-Diagonal Structure II

Block-Diagonal Hessian from Canonical Form

For the partitioned canonical form, because the blocks $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(k)}$ appear independently in the set constraints $\mathbf{x}^{(i)} \in S^{(i)}$, the barrier Hessian $\nabla^2 p_{\text{barrier}}$ is **block-diagonal**:

$$\nabla^2 p_{\text{barrier}}(\mathbf{x}) = \begin{bmatrix} \nabla^2 p_{\text{barrier}}^{(1)}(\mathbf{x}^{(1)}) & & \\ & \ddots & \\ & & \nabla^2 p_{\text{barrier}}^{(k)}(\mathbf{x}^{(k)}) \end{bmatrix}$$

where $\nabla^2 p_{\text{barrier}}^{(i)}$ (“nabla-squared of p-barrier sub- i ”) is the Hessian of the barrier for block i alone, which depends only on $\mathbf{x}^{(i)}$ and not on any other block.

Interior Point Methods and Block-Diagonal Structure III

Notation

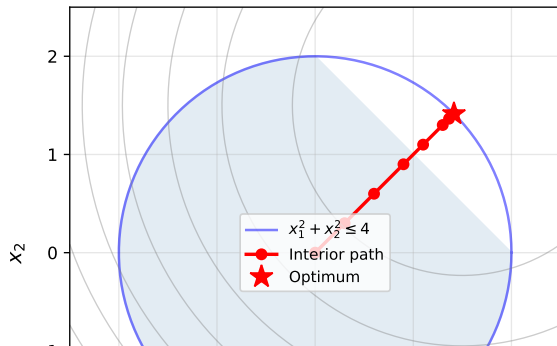
- ∇^2 (“nabla-squared”, from ∇ (nabla, meaning gradient/del)) denotes the Hessian matrix of second partial derivatives of a scalar function. For a function of n variables, it is an $n \times n$ symmetric matrix. Its (i, j) entry is the second derivative with respect to the i -th and j -th variables simultaneously.
- p_{barrier} is the barrier-augmented objective: the sum of the original linear cost and the logarithmic penalty terms for each constraint.
- A matrix is **block-diagonal** if all entries outside the square blocks on the main diagonal are zero. Solving a linear system with a block-diagonal matrix of blocks of sizes $s_1, s_2, \dots, s_k$ costs $\sum_i O(s_i^3)$ operations, which is much cheaper than the $O((\sum_i s_i)^3)$ cost for a fully dense matrix of the same total size.

Interior Point Methods and Block-Diagonal Structure IV

How to Read This Formula

Because the Hessian is block-diagonal, inverting it costs roughly the *sum* of the cost for each block individually, rather than the cost for the full dense matrix. For a problem with k equal-sized blocks of size s , this is a factor of k^2 cheaper — potentially thousands of times faster for large, well-structured problems. This is the computational payoff of DCP canonicalization: it directly enables the solver to exploit the block structure for dramatic speed improvements.

Interior Point Method on Convex QP



The figure illustrates the trajectory of an interior point method: starting from the interior of the feasible region (not at the boundary), the iterates follow a smooth *central path* that curves inward toward the optimum. Because the Hessian is block-diagonal at each step, each Newton step is fast. The method typically converges in 20–80 iterations regardless of

CVXPY: Selecting and Inspecting Solvers

```
import cvxpy as cp
import numpy as np

x = cp.Variable(2)
A = np.eye(2)           # 2x2 identity matrix
b = np.array([1., 2.])  # target vector

# Problem: minimise L2 distance from x to b, subject to x in L1-ball
prob = cp.Problem(
    cp.Minimize(cp.norm(A @ x - b, 2)),  # L2 norm: Euclidean distance
    [cp.norm(x, 1) <= 2.0]               # L1 ball constraint
)

# CVXPY supports multiple solvers; choose based on problem type:
# SCS: open-source, general conic, works well for large problems.
# ECOS: open-source interior-point, very good for small/medium SOCP.
# MOSEK: commercial (free for academics), fastest for LP/SOCP/SDP.

prob.solve(solver=cp.SCS)
print(f"SCS  x* = {x.value}")

prob.solve(solver=cp.ECOS)
print(f"ECOS x* = {x.value}")

# Uncomment to use MOSEK (requires separate license file):
# prob.solve(solver=cp.MOSEK)
```

Chapter Summary I

This chapter developed the complete pipeline for disciplined convex programming, from problem specification all the way through to numerical solution. Each stage of the pipeline addresses one distinct challenge: (1) how to verify that a problem is convex, (2) how to convert it into a form solvers can accept, and (3) how to efficiently solve the resulting structured problem.

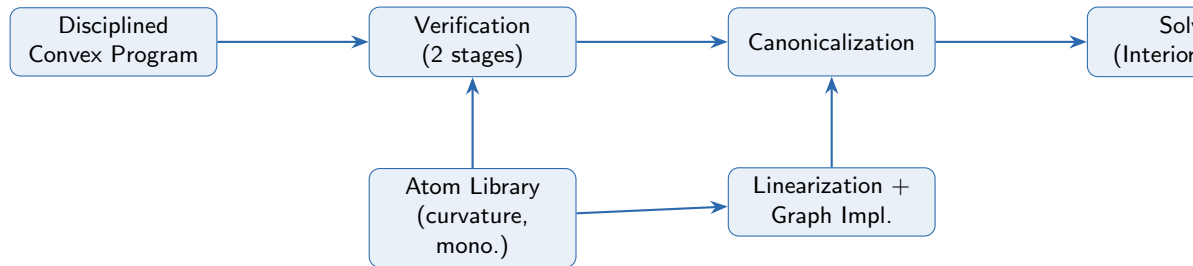
Chapter Summary II

Key Takeaways

- 1 **DCP provides a grammar for convex problems.** By requiring expressions to be product-free compositions of atoms from a curated library, DCP makes convexity checkable automatically by software. The human writes a high-level model; the computer verifies its correctness and translates it to a solver-ready format.
- 2 **Verification is a two-stage tree walk.** Stage 1 checks the algebraic structure (no variable products). Stage 2 propagates curvature labels (convex/concave/affine) up the expression tree using sign, composition, and top-level rules. Both stages run in time linear in the problem size — essentially instantaneous.
- 3 **Canonicalization separates modelling from solving.** Linearization (replacing atom calls with variables plus constraints) and graph implementation (replacing atom constraints with linear or conic ones) convert any valid DCP into a partitioned canonical form. This form has a purely linear objective and block-structured set constraints.
- 4 **Interior point methods exploit canonical structure.** The block-diagonal Hessian of the canonical form dramatically reduces the cost of each Newton step, making large DCP problems tractable. Interior point methods typically converge in 20–80 iterations, with each iteration fast due to the block structure.

Concept Map — How the Chapter Pieces Fit Together I

The following diagram shows how the major concepts of the chapter relate to each other. Reading the diagram left to right traces the complete DCP pipeline from problem formulation to numerical solution.



Concept Map — How the Chapter Pieces Fit Together II

Reading the diagram: the user writes a **Disciplined Convex Program**, which is first **verified** using the **Atom Library** to check that every expression satisfies the curvature grammar. If verification passes, the problem is **canonicalized** using **linearization** and **graph implementations** (both drawing on the atom library for their conic representations), and the resulting partitioned canonical form is handed to an **interior-point solver**. The solver returns the globally optimal solution.

The Central Theme

At every stage, the key principle is the same: separate concerns. The atom library separates “what functions are convex” from “how to check convexity at runtime”. Verification separates “is this valid?” from “how do I solve it?”. Canonicalization separates “expressing the problem” from “choosing a solver”. This separation of concerns is what makes DCP scalable, reliable, and extensible.

Selected Exercises with Guidance I

Exercise 14.1 — Extended-Value Functions

Recall that a function f defined on a convex set $\mathcal{S}$ (“script S ”) can be extended to all of $\mathbb{R}^n$ by setting $f(\mathbf{x}) = +\infty$ for $\mathbf{x} \notin \mathcal{S}$. Define the extended function g formally: $g(\mathbf{x}) = f(\mathbf{x})$ if $\mathbf{x} \in \mathcal{S}$, else $g(\mathbf{x}) = +\infty$. Show that g is convex.

Approach: To prove convexity you must show that for any two points $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and any λ (lambda, pronounced “lam-duh”) $\in [0, 1]$:

$$g(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) \leq \lambda g(\mathbf{x}) + (1 - \lambda) g(\mathbf{y}).$$

Hint: There are three cases. (1) Both $\mathbf{x}$ and $\mathbf{y}$ lie in $\mathcal{S}$: the convex combination $\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}$ also lies in $\mathcal{S}$ (since $\mathcal{S}$ is convex), so apply the convexity of f to get $g(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) \leq \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}) = \lambda g(\mathbf{x}) + (1 - \lambda) g(\mathbf{y})$. (2) Exactly one of $\mathbf{x}, \mathbf{y}$ lies in $\mathcal{S}$: the right-hand side contains $+\infty$, so the inequality holds trivially. (3) Neither lies in $\mathcal{S}$: both sides equal $+\infty$, so the inequality holds with equality.

Exercise 14.2 — Squares in DCP

Products of two variables $x_1 \cdot x_2$ are not DCP-compliant because their curvature cannot be determined from the curvatures of the individual factors alone. However, x^2 is a special case that can be handled by defining a `square(x)` atom with the following properties stored in the atom library: curvature = convex; nondecreasing for $x \geq 0$; nonincreasing for $x \leq 0$; range = $[0, \infty)$.

Using these properties, apply the composition rule to verify that `square(g(x))` is convex when g is affine. Then construct an example where g is concave and determine whether the composition is convex, concave, or neither.

Hint: The sign of $g(x)$ determines which monotonicity branch applies — this is why the range of g matters, not just its curvature.

Selected Exercises with Guidance III

Exercise 14.5 — Graph Implementation of $\sqrt{x}$

The square root function $\sqrt{x}$ is **concave** on $x \geq 0$, so its *hypograph* is used for the graph implementation. The hypograph is:

$$\text{hypo } \sqrt{x} = \{(x, y) \mid x \geq 0, y \leq \sqrt{x}\} = \{(x, y) \mid x \geq 0, y^2 \leq x, y \geq 0\}$$

(squaring both sides of $y \leq \sqrt{x}$ for $y \geq 0$ gives $y^2 \leq x$, which is a second-order cone constraint). The graph implementation is therefore:

$$\sqrt{x} \Rightarrow \arg \max_{y \geq 0} y \quad \text{s.t.} \quad y^2 \leq x$$

Verify numerically in CVXPY that `cp.sqrt(x)` with $x = 4$ returns 2.0. Then verify that `cp.sqrt(cp.square(x))` equals $|x|$ for a few test values.

Selected Exercises with Guidance IV

Exercise 14.6 — Full Canonicalization by Hand

Consider the running problem:

$$\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_1 \quad \text{s.t.} \quad \|\mathbf{Cx} - \mathbf{d}\|_{1.5} \leq 3$$

Perform the full canonicalization by hand.

Step 1 — Linearize: introduce $\mathbf{v}^{(1)} \in \mathbb{R}^2$ for the L_1 objective (one component per component of $\mathbf{Ax} - \mathbf{b}$) and $\mathbf{v}^{(2)} \in \mathbb{R}^2$ for the $L_{1.5}$ constraint.

Step 2 — Graph expand L_1 : $-\mathbf{v}^{(1)} \leq \mathbf{Ax} - \mathbf{b} \leq \mathbf{v}^{(1)}$; the objective becomes $\mathbf{1}^\top \mathbf{v}^{(1)}$.

Step 3 — Graph expand $L_{1.5}$: $|(\mathbf{Cx} - \mathbf{d})_i|^{1.5} \leq v_i^{(2)}$ for $i = 1, 2$, and $\mathbf{1}^\top \mathbf{v}^{(2)} \leq 3$.

Step 4: Write out the complete LP/SOCP and count the total number of variables (original plus all auxiliary) and total number of constraints. Verify your result by solving the problem with CVXPY on the specific numerical data.

CVXPY: Solving Exercise 14.6 Numerically

```
import cvxpy as cp
import numpy as np

# Problem data (same as the running example)
A = np.array([[2., -1.], [1., 3.]])
b = np.array([4., -5.])
C = np.array([[0., 1.], [3., -4.]])
d = np.array([-1., 0.])

x = cp.Variable(2)

# Build the DCP

objective = cp.Minimize(cp.norm(A @ x - b, 1))
constraints = [cp.norm(C @ x - d, 1.5) <= 3]
problem = cp.Problem(objective, constraints)
print("Is DCP:", problem.is_dcp()) # True -- passes verification

# Solve with ECOS (interior-point SOCP solver)
problem.solve(solver=cp.ECOS, verbose=False)

# Report results

print(f"Status : {problem.status}")
print(f"Optimal x : {x.value}") # should be ~[1.23, -0.45]
```

Further Reading and Online Resources I

Primary References

- M. Grant, S. Boyd, Y. Ye, “*Disciplined Convex Programming*,” in *Global Optimization: From Theory to Implementation*, Springer, pp. 155–210, 2006. This is the foundational paper that introduces the DCP framework, formalises the atom library, composition rules, and canonicalization pipeline. Reading this paper gives the mathematical derivation of every algorithm in this chapter.
- S. Diamond, S. Boyd, “*CVXPY: A Python-Embedded Modeling Language for Convex Optimization*,” *Journal of Machine Learning Research (JMLR)*, vol. 17, pp. 1–5, 2016. Describes the design and implementation of CVXPY, including its extensions to mixed-integer and parametric programmes. Read this for a deeper understanding of how CVXPY works internally.
- S. Boyd, L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. The standard reference for convex analysis, duality theory, interior point methods, and applications in signal processing, machine learning, and control. Freely available at cvxbook.stanford.edu.

Further Reading and Online Resources II

Online Resources for Hands-On Practice

- dcp.stanford.edu — an interactive DCP analyser where you can type expressions and immediately see whether they are valid DCP, along with the curvature label assigned to each sub-expression. This is an excellent tool for building intuition about the composition and sign rules.
- cvxpy.org — the official CVXPY documentation, including tutorial Jupyter notebooks, a full reference for all atoms in the Python atom library, and examples from many application domains.
- cvxgrp.stanford.edu — the Stanford Convex Optimization Group, which maintains both CVXPY and the DCP framework. This site includes research papers, course materials, and links to related open-source solvers.

Summary of Chapter 14

The fundamental message of disciplined convex programming is that *expressing a problem in a disciplined way* — using an atom library and a simple grammar — enables *fully automatic verification and solution*. This transforms convex optimisation from a topic for specialists into a practical everyday tool for engineers, data scientists, and operations researchers.